

Page No :-
Date :- / /

→ class Student {

PSum()

$f_{load} = 0.7 f_s$

Sout((double)a); // 0.699998807971 but not 0.7

IEEE 754 floating point repn

1 bit	8 bit	23 bits
Sign	Exponent	Mantissa

9.125 f

Step 2: Convert 97 to Binary

4 → 100

• $12.5 \rightarrow .001$

After Step 1 we get binary as
100.001

Top	0.125×2	0.25	0
↓ to	0.25×2	0.5	0
Bottom	0.5×2	1.00	1

Step 2: Make it in the form of $(1. xxx) \times 2^{\text{exp}}$

$$|00.00| \Rightarrow |1.00001| \times 10^2$$

step 3: add bias to the exponent

for the float 8 bit bias $2^8 - 1 = 127$

$$127 + 2 = 129$$

step 4: fill it

[illegible]

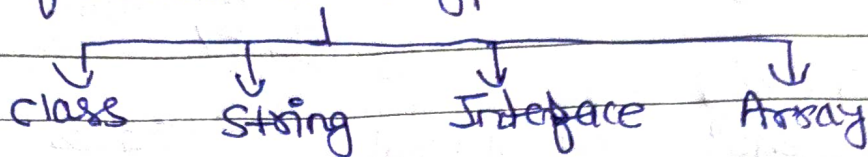
Stored in memory

Address memory
 ↓
 Value conversion
 $(129)_{10} \Rightarrow 10000001$
 $2^{-1} \quad 2^{-2} \quad 2^{-3} \quad 2^{-4} \quad 2^{-5}$
 $00001 = \frac{1}{32} =$
 exp: $(1 + mantissa) \times e^{-127}$

$$\Rightarrow (-1)^0 \times (1 + \frac{1}{32}) 2^{129-127} = 1 \times (\frac{33}{32}) 2^2 = 1 \times \frac{33}{8} = 1 \times \frac{33}{8} = 1.6625$$

Java variables - Part 2

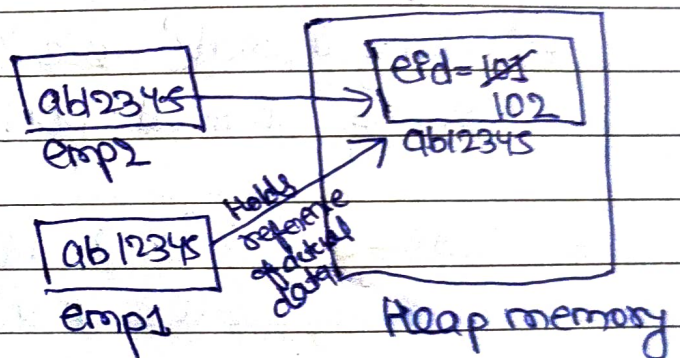
- Reference Data Type (Non Primitive D.T)



Reference datatype holds the reference (memory address) of actual value which is stored in Heap memory.

With the help of reference obj we can change or modify the value in the heap memory.

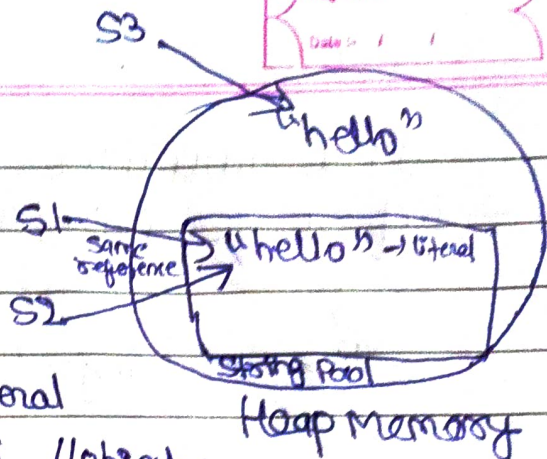
```
class Employee {  
    int eid;  
    Employee(int eid) {  
        this.eid = eid;  
    }  
}
```



```
public class Main {  
    public static void main ( ) {
```

```
        Employee emp1 = new Employee(101);  
        Employee emp2 = emp1;  
        emp2.eid = 102;  
        System.out.println(emp1.eid); // 101
```


String : $\rightarrow$ are immutable
 $\rightarrow$ String literal



String s1 = "hello";
 String s2 = "hello"; // String literal
 String s3 = new String("hello"); // object
 Sout(s1 == s2) // true (because here we check ref not actual value)

Sout(s1 == s3) // false

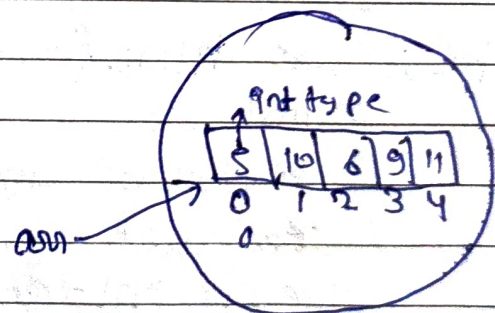
Sout(s1.equals(s3)) // true both have same value
 check content not ref.

- 'String literals' are literals that are present inside String constant pool

Interface : is a blueprint another type of Ref D.T
 we can't create an object of a Interface.

Array :

int[] arr = new int[5];
 arr[0] = 5;
 arr[4] = 11;



Wrapper class
 & primitive type

int
 boolean
 char
 float
 double
 long
 short
 byte

& Reference Type	Wrapper class
Integer	
Boolean	
Character	
Float	
Double	
Long	
Short	
Byte	

- Advantage of wrapper class over primitive DoT
- 1) pass by Reference (because of reference capability)
 - 2) collections work on with object that's why we use wrapper class.

Autoboxing $\Rightarrow$ Primitive to wrapper class
 ex: `int x = 10;`
`Integer a = x;`
`System.out.println(a);` // 10
 // Internally what happens

Unboxing $\Rightarrow$ wrapper class to primitive
`Integer x = 20;`
`int x1 = x;`
 (An arrow points from `x` in the second line to `x` in the first line)

Constant Variable $\Rightarrow$ we make a constant variable with the use of 'Static final'.

ex. class Employee {
 static final int ~~EID~~ = 10;
 }

